

Firestore, de zéro

Mettre la sauvegarde de Cent Saisons en ligne

Ce document explique ce qu'est Firestore, pourquoi le jeu en a besoin, ce que vous avez à faire vous-même (avec les clics, un par un), ce que je ferai dans le code, et ce que cela coûte. Aucune connaissance préalable n'est supposée.

1. Le problème, aujourd'hui

Aujourd'hui, la partie d'un joueur est enregistrée dans son navigateur, dans un espace qui s'appelle le stockage local. Concrètement, c'est un petit fichier caché rangé par le navigateur, sous la clé `cent-saisons.save`.

Ça marche très bien, c'est instantané, ça fonctionne sans connexion. Mais ce stockage a trois défauts, et ce sont les trois raisons de ce document.

- **Il est attaché à un appareil.** Une partie commencée sur le téléphone n'existe pas sur l'ordinateur, et inversement.
- **Il est fragile.** Vider le cache du navigateur, ou passer en navigation privée, efface tout.
- **Il est attaché à un navigateur.** Chrome et Safari sur le même téléphone ne partagent rien.

Le jeu sait déjà télécharger un fichier de sauvegarde et le recharger, et il rappelle régulièrement de le faire. C'est une roue de secours efficace, mais qui demande au joueur d'y penser. Firebase enlève ce souci.

2. Ce qu'est Firebase, en une page

Firebase est un ensemble de services hébergés par Google. Dit simplement : **ce sont des ordinateurs de Google que votre jeu peut utiliser**, sans que vous ayez à louer, installer ou entretenir quoi que ce soit.

Firebase propose une vingtaine de services. Nous n'en utiliserons que deux.

Le mot	Ce que ça veut dire	On s'en sert ?
Projet	La boîte qui contient tout le reste. On en crée un, appelé par exemple « cent-saisons ».	Oui
Authentication	Le service qui sait qui est le joueur. Il sait faire des comptes avec mot de passe, mais aussi, et c'est ce qui nous intéresse, des comptes anonymes : le joueur ne s'inscrit pas, ne donne ni nom ni adresse, et reçoit quand même un identifiant invisible qui lui appartient.	Oui
Firestore	Une base de données. Imaginez un grand classeur en ligne : chaque joueur y a une fiche, et la fiche contient sa sauvegarde.	Oui
Hosting	Un hébergement de site web. Nous gardons GitHub Pages, qui marche déjà.	Non
Storage, Functions, Analytics...	Fichiers, code serveur, statistiques de fréquentation.	Non

Autrement dit, tout ce que nous ajoutons au jeu tient en une phrase : **le joueur reçoit un identifiant anonyme, et sa sauvegarde est rangée en ligne sous cet identifiant.**

3. Deux façons d'entrer : anonyme, puis Google

Cent Saisons est un jeu calme, qu'on ouvre et qu'on joue. Demander une adresse et un mot de passe avant la première tuile serait un mur. Nous proposerons donc les deux, dans cet ordre.

	Anonyme (par défaut)	Google (proposé, jamais imposé)
Quand	Dès la première seconde, sans rien demander.	Quand le joueur le veut, depuis les Options ou après une île.
Ce que le joueur donne	Rien.	Son adresse Google et son prénom.
Ce qu'il y gagne	Sa partie est protégée du vidage du cache, sur cet appareil.	Sa partie le suit sur tous ses appareils, pour toujours.
Le risque	Il perd tout s'il change de téléphone.	Aucun : c'est le cas confortable.

Le passage de l'un à l'autre est la partie élégante : Firebase sait **rattacher** un compte anonyme existant à un compte Google. Le joueur a joué dix îles sans compte, il appuie sur « Continuer avec Google », et **ses dix îles restent** — c'est le même joueur, qui a simplement donné son nom.

Le cas délicat, et il arrivera. Un joueur a une partie anonyme sur son téléphone, et son compte Google porte déjà une partie faite sur l'ordinateur. Le rattachement est alors impossible : il y a deux parties pour un seul joueur. Le jeu affichera les deux, avec leur date, leurs étoiles et leur nombre d'îles, et demandera laquelle garder. C'est la question n° 1 de la page « Décisions ».

Une précision technique qui a des conséquences visibles : sur téléphone, la fenêtre surgissante de Google est souvent bloquée par le navigateur. J'utiliserai donc la méthode par redirection, qui quitte brièvement le jeu et y revient. C'est normal et sans danger, mais il faut que le jeu sache reprendre sa partie au retour : je m'en occupe.

4. Ce que cela coûte

Firebase a un palier gratuit, appelé **Spark**, sans carte bancaire. Voici ses limites pour les deux services qui nous concernent, et ce que notre jeu consomme réellement.

Ce qui est compté	Offert par jour	Ce que consomme une partie	Marge
Écritures dans la base	20 000	1 écriture par île terminée, soit 5 à 15 par session	très large
Lectures dans la base	50 000	1 lecture au lancement du jeu	très large
Espace de stockage	1 Go	environ 4 Ko par joueur	250 000 joueurs
Connexions (anonymes et Google)	sans limite	1 par joueur, puis 1 au rattachement	—

En clair : avec quelques centaines de joueurs, vous resterez très loin des limites, et **vous ne paierez rien**. Le palier gratuit ne bascule pas en payant tout seul : si une limite est atteinte, le service s'arrête jusqu'au lendemain plutôt que de vous facturer. Je prévoirai le cas dans le code : si l'enregistrement en ligne échoue, le jeu continue et garde la sauvegarde locale.

5. Ce que vous avez à faire, pas à pas

Comptez vingt minutes. Tout se fait dans un navigateur, sur `console.firebase.google.com`, avec un compte Google ordinaire. Rien à installer.

1

Créer le projet

Ouvrez `console.firebase.google.com` et connectez-vous avec votre compte Google.

- Cliquez sur **Créer un projet**.
- Nom du projet : **cent-saisons**. Firebase y ajoutera des chiffres pour le rendre unique, c'est normal.
- Google Analytics : **désactivez-le**. Nous n'en avons pas besoin, et cela évite une question de consentement aux cookies.
- Cliquez sur **Créer le projet**, patientez une minute.

2

Déclarer le jeu comme application web

Firebase a besoin de savoir que le jeu est un site web, pour lui donner ses clés.

- Sur la page d'accueil du projet, cliquez sur l'icône `</>` (« Web »).
- Surnom de l'application : **Cent Saisons**.
- Ne cochez pas « Firebase Hosting » : nous gardons GitHub Pages.
- Cliquez sur **Enregistrer l'application**.

Firebase affiche alors un bloc de code contenant une **configuration**. C'est ce bloc qu'il me faut. Il ressemble à ceci :

```
const firebaseConfig = {
  apiKey: "AIzaSy...",
  authDomain: "cent-saisons-1234.firebaseio.com",
  projectId: "cent-saisons-1234",
  storageBucket: "cent-saisons-1234.appspot.com",
  messagingSenderId: "123456789012",
  appId: "1:123456789012:web:abc123def456"
};
```

Copiez ce bloc et gardez-le : vous me le donnerez à l'étape 7.

3

Activer les deux méthodes de connexion

- Dans le menu de gauche : **Créer**, puis **Authentication**, puis **Commencer**.
- Onglet **Sign-in method** (méthodes de connexion).
 - a. **Anonyme**. Dans la liste, choisissez **Anonyme**, basculez sur **Activer**, puis **Enregistrer**.
 - b. **Google**. Dans la même liste, choisissez **Google**, basculez sur **Activer**. Firebase demande alors deux choses :
 - **Nom public du projet** : écrivez **Cent Saisons**. C'est le nom que verra le joueur dans la fenêtre de Google : « Cent Saisons souhaite accéder à votre compte ». Ne laissez pas le nom technique.
 - **Adresse e-mail d'assistance** : choisissez la vôtre dans la liste. Elle sera visible par les joueurs sur cet écran de Google, c'est une obligation de sa part.

Puis **Enregistrer**. Laissez toutes les autres méthodes désactivées.

Firestore configure tout seul, en arrière-plan, ce qu'on appelle l'écran de consentement OAuth. Vous n'avez rien à faire dans la console Google Cloud tant que le jeu reste en **mode test**, ce qui suffit largement pour des centaines de joueurs. Si un jour le jeu grandit beaucoup, il faudra passer cet écran en « production », une formalité de quelques minutes que je vous signalerai.

4

Créer la base de données

- Menu de gauche : **Créer**, puis **Firestore Database**.
- Cliquez sur **Créer une base de données**.
- Emplacement : choisissez **eur3 (europe-west)** ou **europe-west1**. C'est définitif, et l'Europe est le bon choix pour des joueurs français.
- Mode : choisissez **Démarrer en mode production**. Nous écrirons les bonnes règles à l'étape suivante.

5

Écrire les règles de sécurité

C'est l'étape la plus importante du document. Les règles décident qui a le droit de lire et d'écrire quoi. Sans elles, n'importe qui pourrait lire ou effacer les sauvegardes de tout le monde.

- Dans **Firestore Database**, ouvrez l'onglet **Règles**.
- Effacez tout le contenu et collez le texte ci-dessous.
- Cliquez sur **Publier**.

```
rules_version = "2";
service cloud.firestore {
  match /databases/{database}/documents {
    // chaque joueur ne touche que sa propre fiche, et seulement connecte
    match /parties/{joueur} {
      allow read:    if request.auth != null && request.auth.uid == joueur;
      allow delete:  if request.auth != null && request.auth.uid == joueur;
      allow create, update: if request.auth != null && request.auth.uid == joueur
        && request.resource.data.save is string
        && request.resource.data.save.size() < 200000;
    }
    // tout le reste est interdit
    match /{document=**} { allow read, write: if false; }
  }
}
```

Ces règles disent : **un joueur connecté lit, écrit et efface sa fiche, la sienne seulement, et la sauvegarde qu'elle contient ne peut pas dépasser 200 Ko**. Tout le reste est refusé.

Pourquoi les trois cas sont-ils séparés ? Parce que `request.resource` n'existe que pour une écriture. Sur une lecture ou un effacement il vaut *null*, et une règle qui s'appuie dessus échoue — donc refuse tout. Écrites en une seule ligne, ces règles laissent passer l'écriture et bloquent la lecture : la partie part en ligne mais ne revient jamais. Le dépôt contient un test (`tests/firestore_rules.mjs`) qui joue ces règles dans l'émulateur Firebase et vérifie les seize cas, bons et mauvais.

6

Autoriser le domaine du jeu

Firebase n'accepte les connexions que depuis des adresses que vous avez déclarées.

- **Authentication**, onglet **Settings** (paramètres), section **Domaines autorisés**.
- `localhost` y est déjà : gardez-le, il sert à mes essais.
- Ajoutez l'adresse publique du jeu, celle de GitHub Pages, de la forme `votre-nom.github.io`.

7

Me transmettre la configuration

Envoyez-moi le bloc copié à l'étape 2. Vous pouvez le coller directement dans notre conversation.

Ces clés ne sont pas des mots de passe. Elles sont visibles par tout le monde dans n'importe quelle application web utilisant Firebase, y compris les plus grosses : c'est prévu ainsi. Ce qui protège vos données, ce sont les **règles de l'étape 5**, pas le secret des clés. Il n'y a donc aucun risque à me les donner, ni à les mettre dans le dépôt du jeu.

6. Ce que je ferai, moi, dans le code

Dès que j'aurai la configuration, voici le travail, dans l'ordre. Rien de tout cela ne vous demande d'intervenir.

Étape	Ce que ça change pour le joueur
Brancher Firebase — un fichier <code>src/core/cloud.js</code> , chargé seulement quand il y a du réseau.	Rien de visible.
Connexion anonyme au lancement , en silence.	Rien de visible.
Envoyer la sauvegarde à la fin de chaque île, et au retour au menu.	Une petite icône de nuage confirme que la partie est en lieu sûr.
Relire la sauvegarde au lancement, et garder la plus récente entre celle de l'appareil et celle en ligne.	Le joueur retrouve sa partie.
Gérer le conflit : si les deux sauvegardes diffèrent nettement, le jeu demande laquelle garder, avec la date et le nombre d'étoiles de chacune.	Un écran, rarement.
Le bouton « Continuer avec Google » dans les Options, et une proposition discrète après quelques îles. Rattachement du compte anonyme, par redirection sur téléphone.	Un bouton, et sa partie le suit partout.
L'écran des deux parties : quand le compte Google porte déjà une sauvegarde, les deux sont affichées avec leur date, leurs étoiles et leurs îles, et le joueur choisit.	Un écran, rarement.
Se déconnecter , et effacer mes données en ligne : deux boutons dans les Options, pour que le joueur garde la main.	Deux boutons dans les Options.
Tout rendre facultatif : sans réseau, ou si Firebase répond mal, le jeu fonctionne exactement comme aujourd'hui.	Aucune panne visible.
Tests : un simulateur Firebase local dans la suite de tests, pour vérifier sans toucher à la vraie base.	Rien de visible.

Le fichier de sauvegarde à télécharger reste en place. Il ne coûte rien à garder, et il restera la meilleure roue de secours.

7. Trois décisions à prendre

Vous avez déjà tranché la première question du document précédent : ce sera **anonyme par défaut, Google en option**. Restent ces trois points, que je ne peux pas décider à votre place.

Question 1 — Que fait-on quand il y a deux parties pour un joueur ?

Le cas : partie anonyme sur le téléphone, et partie déjà en ligne sur le compte Google.

- **Demander au joueur** (ma recommandation) : un écran montre les deux, avec la date, les étoiles et le nombre d'îles.
- **Garder la plus avancée** automatiquement, sans rien demander. Plus simple, mais un joueur peut perdre une partie qu'il voulait.

Question 2 — Quand propose-t-on la connexion Google ?

- **Discrètement, après la troisième île** (ma recommandation) : un bandeau qui se referme, plus le bouton toujours présent dans les Options.
- **Seulement dans les Options** : le joueur doit y penser. Plus sobre, beaucoup moins utilisé.
- **Dès le menu d'accueil** : efficace, mais c'est le mur qu'on voulait éviter.

Question 3 — Voulez-vous un classement en ligne ?

- **Non pour l'instant** (ma recommandation) : l'île du jour a déjà son meilleur score local, et un classement change le ton du jeu.
- **Oui** : c'est faisable — avec Google, on a même déjà un prénom — mais il faudra une modération et une protection contre la triche. Comptez un lot entier.

8. Ce à quoi il faut faire attention

- **L'emplacement de la base est définitif.** Choisissez l'Europe à l'étape 4 : on ne peut pas le changer ensuite sans tout recréer.
- **Ne laissez jamais les règles en mode test.** Firebase propose un mode où tout est ouvert pendant trente jours. Les règles de l'étape 5 évitent ce piège.
- **Ne collez pas votre configuration dans un ticket public** par habitude : ce n'est pas dangereux, mais autant rester discret.
- **Le jeu doit continuer de marcher hors ligne.** C'est une règle que je m'impose : Firebase est un confort, jamais une dépendance.
- **Vie privée : la connexion Google change la donne.** Une adresse e-mail est une donnée personnelle. Il faudra donc une courte page « Vie privée » dans le jeu, disant quoi est collecté (adresse, prénom, sauvegarde), pourquoi (retrouver sa partie), où (Google, en Europe), et comment tout effacer. Je l'écrirai et j'ajouterai le bouton « Effacer mes données en ligne ». Rien de lourd : quinze lignes et un bouton.
- **Le nom public du projet est visible par les joueurs** dans la fenêtre de Google. Vérifiez qu'il dit bien « Cent Saisons » et non « cent-saisons-1234 ».

9. En résumé

Vous : vingt minutes dans la console Firebase, sept étapes, puis vous m'envoyez le bloc de configuration et vos réponses aux trois questions.

Moi : je branche le tout, je teste, et j'écris la page « Vie privée ». Le joueur commence sans rien donner, et le jour où il appuie sur « Continuer avec Google », sa partie le suit partout, sans perdre une seule île.

Le coût : zéro, très largement, et sans carte bancaire.